

Barcode Scanning Technologies for the Food Recall App

One of the core features of the Food Recall App is the ability to scan product barcodes using a mobile device. This function enables users to check for food recalls quickly and efficiently, reducing the risk of exposure to unsafe products. To implement this feature, we evaluated several barcode scanning libraries compatible with **React Native**, focusing on performance, ease of integration, platform support (iOS and Android), accuracy, documentation quality, and licensing.

Below is a comparison of the top three barcode scanning libraries considered:

Software	Pros	Cons
Google ML Kit Barcode API	- High accuracy with support for multiple formats - Active Google support - Easy integration via Firebase - Offline capability	- Some features require Firebase setup - Slightly larger app size
ZXing (Zebra Crossing)	- Open source and well-established - Good format support - Lightweight	- Integration with React Native requires third-party wrappers - Less accurate than ML Kit
Dynamsoft Barcode Reader	- Extremely fast and accurate - Commercial-grade quality - Extensive format support	- Paid license required for full features - Limited community support

Justification for Selected Technology

After careful consideration, we selected **Google ML Kit Barcode Scanning API** as the barcode scanning technology for the Food Recall App. It offers a strong balance between accuracy, offline capabilities, cross-platform support, and ease of integration with React Native via the *react-native-google-mlkit* package. Google ML Kit supports a wide range of barcode types, including QR codes, UPC, and EAN formats, which are crucial for grocery item recognition.

While **ZXing** is open-source and has been around for many years, its integration with React Native requires third-party libraries like *react-native-camera*, which adds extra complexity and maintenance overhead. Additionally, it doesn't match the detection speed or accuracy of ML Kit. **Dynamsoft**, while highly accurate and enterprise-grade, was ruled out due to its commercial licensing cost, which doesn't align with our current budget constraints for the MVP phase.

Our choice of **Google ML Kit** aligns well with our existing technology stack, particularly since we are building the app in **React Native**, and the ML Kit barcode module is supported via a well-maintained wrapper. The decision was based on a combination of developer experience, technical documentation, and community support.

Works Cited

- Google ML Kit Barcode Scanning API Documentation. Google Developers. <https://developers.google.com/ml-kit/vision/barcode-scanning>
- ZXing Project Page. GitHub. <https://github.com/zxing/zxing>
- Dynamsoft Barcode Reader SDK. <https://www.dynamsoft.com/barcode-reader/overview/>
- react-native-google-mlkit. GitHub. <https://github.com/react-native-ml-kit/barcode-scanning>